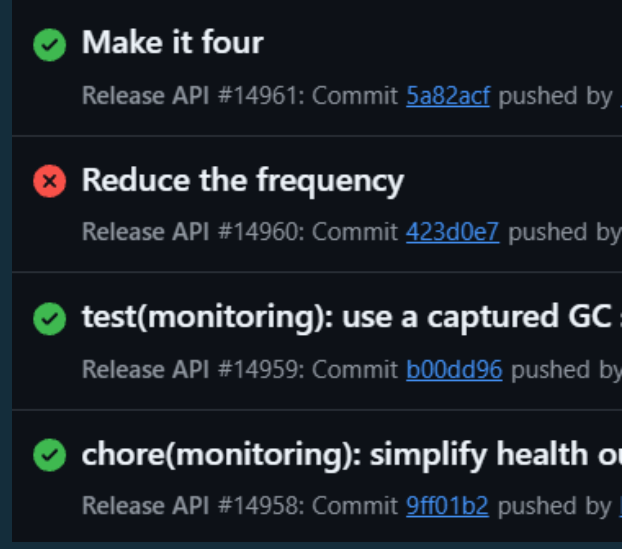
Обчислювальна математика та програмування

Юніт-тести у Python

Як перевіряти поведінку програми

Практичне заняття №3 · 40 хвилин пояснення




```python
import importlib
import sys
import pathlib

ROOT = pathlib.Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
combined = importlib.import_module("ControlBlocks.main")

ControllerState = combined.ControllerState
JoystickValue = combined.JoystickValue
pack_state = combined.pack_state
ControllerStateParser = combined.ControllerStateParser


def test_serial_roundtrip():
    state = ControllerState(
        left=JoystickValue(x=1, y=-2, boost=3, press=True, speed_x=100, speed_y=200),
        right=JoystickValue(x=-5, y=6, boost=7, press=False, speed_x=300, speed_y=400),
    )
    packed = pack_state(state)
    parser = ControllerStateParser()
    result = parser.feed(packed)
    assert len(result) == 1
    restored = result[0]
    assert restored.left.x == state.left.x
    assert restored.left.y == state.left.y
    assert restored.right.boost == state.right.boost
    assert restored.right.speed_y == state.right.speed_y

```

Перевірки з попереднього заняття

Похибка потребує критерію порівняння.

Для матриці перевіряємо значення та форму.

Поелементна функція має область визначення.

Чистий запуск прибирає прихований стан notebook.

Філософія юніт-тестів

Невелика одиниця поведінки має явне очікування.

Перевірку можна повторити після кожної зміни.

Успіх стосується перевірених сценаріїв.



Фотографія виглядає нормально, баг залишається

250 + 20 = 270

Обмеження яскравості повертає 255.

На великому фото легко пропустити кілька хибних пікселів.

Факт: максимум 8-бітного беззнакового значення — 255.

Контракт функції та тестовий оракул

$p \in \{0, \ldots, 255\}, \quad d \in \mathbb{Z}$

$f(p, d) = \min(255, \max(0, p + d))$

Тестовий випадок: (p, d, y); перевірка: f(p, d) = y

Python Challenge: очікування до запуску

Вхід: (100, +20), (250, +20), (5, −20).

Результат: таблиця очікувань із поясненнями.

Контекст: зміна яскравості без виходу за [0, 255].

Перший тест і його запуск

Arrange: підготувати конкретні дані.

Act: один раз виконати потрібну дію.

Assert: зіставити результат з очікуванням.

pytest: автоматичний запуск перевірок

Функція живе в image_ops.py.

Перевірки живуть у test_image_ops.py.

pytest знаходить тести й формує звіт.

Факт: звичайний assert дає розбір невдалого порівняння.

Один тест як виконуване твердження

```
from image_ops import brighten

def test_brighten_increases_brightness():
    pixel = 100
    delta = 20

    result = brighten(pixel, delta)

    assert result == 120
```

Критерій успіху: t(f) = PASS ⇔ f(100, 20) = 120.

Результат запуску й причина невдачі

```
FAILED test_image_ops.py::
    test_brighten_increases_brightness

> assert result == 120
E assert 121 == 120

1 failed
```

Запуск: python -m pytest -q test_image_ops.py
Очікуваний дефект: функція повернула 121 замість 120.

Python Challenge: перший запуск

Вхід: готова функція та один тест із прикладу.

Дія: успішний запуск і навмисно хибне очікування.

Результат: пояснення повідомлення FAILED.

Вибір тестових випадків

Звичайний випадок перевіряє основний сценарій.

Межі діапазону перевіряють переходи між правилами.

Кожен випадок має пояснювати можливу помилку.

Дефект, який проходить перший тест

```
def brighten(pixel, delta):
    return min(255, pixel + delta)

# Навмисний дефект: немає нижньої межі
```

Для (100, 20) отримуємо 120.
Для (5, −20) отримуємо −15 замість 0.
Практичний факт: один успішний сценарій не охоплює всі гілки.

Класи входів і граничні значення

p	d	Очікуємо	Сценарій
100	+20	120	Звичайний
250	+20	255	Верхнє насичення
5	−20	0	Нижнє насичення
0	0	0	Нижня межа
255	0	255	Верхня межа
0	+1	1	Крок від нуля
255	−1	254	Крок від 255

s = p + d; очікування визначаємо до запуску.

Одна перевірка, кілька наборів даних

```
import pytest
from image_ops import brighten

@pytest.mark.parametrize("p,d,expected", [
    (100, 20, 120),
    (250, 20, 255),
    (5, -20, 0),
])
def test_brighten_cases(p, d, expected):
    assert brighten(p, d) == expected
```

parametrize створює окремий запуск для кожного рядка.
У звіті видно, який набір спричинив невдачу.

Python Challenge: власний набір випадків

Вхід: контракт і таблиця класів.

Результат: тести для звичайних та граничних входів.

Власний випадок: обґрунтувати, який дефект він шукає.

Тести під час зміни коду

Регресія порушує раніше правильну поведінку.

Тест зберігає конкретну домовленість.

Рефакторинг змінює структуру зі збереженням поведінки.

Red, Green, Refactor

Етап	Дія	Перевірка
Red	Додати сценарій	Падає з потрібної причини
Green	Виправити поведінку	Проходить увесь набір
Refactor	Покращити структуру	Поведінка зберігається

Практичний кейс: виправлення нижнього насичення.
Факт: TDD починає короткий цикл із невдалого тесту.

Критерії якості тестового набору

Очікування визначаємо незалежно від реалізації.

Результат не залежить від порядку запуску.

Для внесеного дефекту має існувати тест, що впаде.

$\exists\ t \in T : t(f) = \text{PASS},\ \ t(f') = \text{FAIL}$

Python Challenge: чи помічають тести баги

Вхід: правильна функція та власний набір тестів.

Експеримент: окремо внести три погоджені дефекти.

Результат: таблиця «дефект — тест — причина невдачі».

Задачі для самостійної роботи

01 Складіть таблицю очікувань для звичайної зміни яскравості та обох насичень, обґрунтувавши відповіді контрактом.

02 Запустіть готовий тест, навмисно змініть очікування та поясніть причину невдачі після повторного запуску.

03 Доповніть набір граничними й власними випадками та поясніть призначення кожного з них.

04 Перевірте набір трьома навмисними дефектами, зафіксуйте виявлені порушення й відновіть правильний код.

Summary / Ключові концепції лекції

Контракт і незалежне очікування задають зміст перевірки.

Arrange, Act, Assert перетворюють сценарій на тест.

Граничні випадки перевіряють різні правила поведінки.

Регресійні тести супроводжують виправлення й рефакторинг.